Laboratory Exercise 5: Data Persistence, Exception Handling & Object-Oriented Programming

Student Details	
Student Name: _____	**Student ID:** _____
Duration: 2 Hours (120 Minutes)	**Total Points:** 100 Points

1. Learning Objectives

- **File I/O & Exception Handling:** Read/write files using context managers (`with open`) and defensively handle exceptions (`try – except – finally`).
- **JSON Serialization:** Serialize and deserialize complex nested data structures to/from disk using `json.dump()` and `json.load()` .
- **Object-Oriented Programming:** Define classes with `__init__` , encapsulate state/methods, and manage multi-object fleet dispatching.

2. Exercise Structure, Evaluation & Instructor Sign-Off

This 120-minute laboratory session is divided into 5 sequential parts totaling 100 points: **Part 1: Robust File I/O & Defensive Exception Handling** (20 mins, 15 pts), **Part 2: JSON Data Interchange & State Serialization** (25 mins, 20 pts), **Part 3: Object-Oriented Architecture: Classes & Attributes** (25 mins, 20 pts), **Part 4: Practical Modeling: `Aircraft` Flight Telemetry Engine** (30 mins, 25 pts), and **Part 5: Challenge: Flight Operations Manager (OOP + JSON + Exceptions)** (20 mins, 20 pts).

Instructor Verification	Details
Score Awarded: _____ / 100 Points	**Instructor Signature:** _____ **Date:** _____ / _____ / 20__
Instructor Comments & Feedback:	_____

Part 1: Robust File I/O & Defensive Exception Handling (15 Points)

Task 1.1: Flight Log Writer & Reader (8 Points)

Write a Python script that:

1. Opens (create) a file named `flight_log.txt` in write mode (`"w"`) using a `with` statement.
2. Writes 3 lines of comma-separated waypoint entries:

```text
WP01,10:15,FL180,250
WP02,10:35,FL280,310
WP03,11:05,FL350,420
```

3. Re-opens `flight_log.txt` in read mode (`"r"`), loops through each line, and prints each entry cleanly formatted without extra blank lines.

Task 1.2: Defensive Exception Handling (7 Points)

Write a function `safe_read_altitude(filename)` that:

1. Attempts to open and read an altitude value from a user-specified filename.
2. Catches `FileNotFoundError` if the file does not exist and displays a user-friendly error: `"[ERROR] Specified flight file not found on disk."` .
3. Catches `ValueError` if the content cannot be converted to a float and displays: `"[ERROR] Corrupted altitude record; non-numeric data encountered."` .
4. Uses the `finally` block to display: `"[LOG] File operation attempt concluded."` .

Student Answer / Program Output:

- _____
  _____

Part 2: JSON Data Interchange & State Serialization (20 Points)

Task 2.1: Exporting Flight Dispatch Plan to JSON (10 Points)

Given the following flight operational plan:

```python
dispatch_plan = {
    "flight_number": "TG920",
    "origin": "VTBS",
    "destination": "EDDF",
    "aircraft": "Airbus A350-900",
    "cruising_alt_ft": 38000,
```

```
    "passengers": 284,
    "fuel_required_kg": 46500.0,
    "alternate_airports": ["EDDM", "EDDL"]
}
```

Write Python code using the `json` module to:

1. Serialize and save `dispatch_plan` into a file named `flight_plan.json` with an indentation of 4 spaces (`indent=4`).
2. Read `flight_plan.json` back into a new variable `loaded_plan` .
3. Access and display the destination airport code and the list of alternate airports.

Student Answer / Program Output:

- **JSON File Generated:** `flight_plan.json`
- **Destination Airport Code:** _____
- **Alternate Airports List:** _____

Task 2.2: Updating JSON Records (10 Points)

Write code to:

1. Append `"EDDK"` into the `alternate_airports` list inside `loaded_plan` .
2. Update `"fuel_required_kg"` by adding a contingency reserve of `2,500.0 kg` .
3. Overwrite the updated plan back to `flight_plan.json` .

Student Answer / Program Output:

- **Updated `alternate_airports` :** _____
- **Updated `fuel_required_kg` :** _____ kg

Part 3: OOP Fundamentals — Classes & Instance Variables (20 Points)

Task 3.1: Defining the `Airport` Class (10 Points)

Define a class named `Airport` that represents a civil aerodrome:

1. Define the constructor `__init__(self, icao_code, name, elevation_ft, num_runways)` .
2. Initialize corresponding instance attributes: `self.icao_code` , `self.name` , `self.elevation_ft` , and `self.num_runways` .
3. Add a method `get_info(self)` that returns a formatted descriptive string:
    - Example: `"Suvarnabhumi Airport (VTBS) | Elevation: 5 ft | Runways: 2"`

Task 3.2: Object Instantiation (10 Points)

Create two distinct instances of the `Airport` class:

- Airport 1: `icao="VTBS"` , `name="Suvarnabhumi Airport"` , `elevation=5.0` , `runways=2`

- Airport 2: `icao="VTCC"`, `name="Chiang Mai International"`, `elevation=1037.0`, `runways=1`

Call and print the `.get_info()` method for both objects. Demonstrate that modifying Airport 1 does not alter Airport 2.

Student Answer / Program Output:

- **Airport 1 Info (`.get_info()`):**_____
- **Airport 2 Info (`.get_info()`):**_____

Part 4: Practical Modeling: `Aircraft` Flight Telemetry Engine (25 Points)

Design an engineering class named `Aircraft` that models dynamic flight physics and fuel consumption.

Class Specification:

- **Constructor `__init__(self, callsign, model, fuel_kg, cruise_speed_kts)`:**
  - Sets initial instance attributes:
    - `self.callsign = callsign`
    - `self.model = model`
    - `self.fuel_kg = float(fuel_kg)`
    - `self.cruise_speed_kts = float(cruise_speed_kts)`
    - `self.altitude_ft = 0.0` (starts on the ground)
    - `self.is_airborne = False`
- **Method `takeoff(self, target_altitude_ft)`:**
  - Checks if `self.fuel_kg < 1000.0`. If so, print `"[ABORT] Insufficient fuel for takeoff!"` and return `False`.
  - Otherwise, sets `self.is_airborne = True`, `self.altitude_ft = target_altitude_ft`, deducts `800.0 kg` of fuel for the climb phase, and prints a success takeoff message.
- **Method `fly(self, hours)`:**
  - If the aircraft is not airborne, print `"[ERROR] Aircraft cannot cruise while on ground."`
  - Computes fuel burn during cruise: $\text{fuel_burn} = \text{hours} \times 2400.0 \text{ kg/hr}$.
  - If $\text{fuel_burn} > \text{self.fuel_kg}$, trigger an emergency: deduct all fuel to 0, set `self.altitude_ft = 0`, and print `"[EMERGENCY] Fuel exhaustion!"`.
  - Otherwise, deducts fuel_used from `self.fuel_kg` and displays distance covered: $\text{distance} = \text{hours} \times \text{self.cruise_speed_kts}$.

- Method `get_status(self)`:
  - Displays a clean formatted status report showing callsign, airborne status, current altitude, and remaining fuel.

Test Workflow:

1. Instantiate: `plane = Aircraft("THA911", "Boeing 777-300ER", fuel_kg=12500.0, cruise_speed_kts=480.0)`.
2. Execute `plane.takeoff(target_altitude_ft=34000.0)`.
3. Execute `plane.fly(hours=3.5)`.
4. Display `plane.get_status()`.

Example Output:

```text
[TAKEOFF] THA911 (Boeing 777-300ER) climbed to FL340. Fuel burn: 800.0 kg.
[CRUISE] THA911 cruised for 3.5 hrs. Distance: 1680.0 NM | Fuel burned: 8400.0 kg.

--- Aircraft Telemetry Status ---
Callsign: THA911 (Boeing 777-300ER)
Airborne: True
Current Altitude: 34,000 ft
Remaining Fuel: 3,300.0 kg
```

Student Answer / Program Output:

- **Climb Phase Fuel Burned:** _____ kg
- **Cruise Distance Covered:** _____ NM
- **Cruise Fuel Burned:** _____ kg
- **Remaining Fuel:** _____ kg

Part 5: Challenge Task — Flight Dispatch Manager (20 Points)

Combine **OOP**, **File I/O**, and **Exception Handling** by creating a class `FlightDispatcher`:

Requirements:

1. `Instantiate:`
   1. `plane1 = Aircraft("THA911", "Boeing 777-300ER", fuel_kg=12500.0, cruise_speed_kts=480.0).`
   2. `plane2 = Aircraft("BKP202", "ATR 72-600", fuel_kg=4500.0, cruise_speed_kts=280.0)`
2. **Attributes:**
   - `self.active_flights = []` (list of `Aircraft` objects).
3. Method `add_aircraft(self, aircraft_obj)`:
   - Appends an `Aircraft` object to the active fleet.
4. Method `save_fleet_manifest(self, filepath)`:

- Loops through `self.active_flights` and builds a list of dictionaries capturing each plane's state (`callsign` , `model` , `altitude_ft` , `fuel_kg` , `is_airborne`).
- Saves the list as JSON into `filepath` using `try – except` to catch any `IOError` .

5. **Method `load_fleet_manifest(self, filepath)` :**
   - Loads the JSON file using `with open(...)` inside a `try – except` block.
   - Recreates `Aircraft` instances and repopulates `self.active_flights` .

Example Run:

```text
[DISPATCH] Fleet manifest exported to 'fleet_manifest.json' (2 aircraft).
[DISPATCH] Fleet manifest successfully reloaded from disk.
Active Track 1: THA911 | Alt: 34000 ft | Fuel: 3300.0 kg
Active Track 2: BKP202 | Alt: 0 ft | Fuel: 4500.0 kg
```

Student Answer / Program Output:

- **Manifest Export Message: _____**
- **Manifest Reload Message: _____**
- **Active Track 1 Status: _____**
- **Active Track 2 Status: _____**